

دليل مهندس البرمجيات

by Yehia Tech



من كتابة الكود إلى بناء أنظمة تنجح في Production.



المحتويات

من هو مهندس البرمجيات؟	01
ليه تعريف Software Engineer بقى محتاج يتراجع؟	02
قابل أحمد... وباسل.	03
نفس ال Task ... عقلية مختلفة.	04
الفرق في طريقة التفكير.	05
Coding ≠ Software Engineering	06
طب أدرس إيه؟	07
Problem & Requirements — بنحل إيه أصلاً؟	08
System Design مش رسمة Interview.	09
Data & Databases — البيانات تعيش إزاي؟	10
Reliability — ماذا لو فشل؟	11
Security — مين مسموح له يعمل إيه؟	12
Performance & Cost — هل الحل ينفع يكبر؟	13
Production Engineering — ال Deploy بداية مسؤولية.	14
Testing & Change — نغيّر بأمان إزاي؟	15
وال AI يدخل فين؟	16
أدرسهم بأي ترتيب؟	17
Software Engineer Checklist	18
ال AI بيرفع سقف مهندس البرمجيات.	19

مهندس البرمجيات؟

مهندس البرمجيات **يفهم المشكلة**، يصمم النظام المناسب، يوازن بين الـ Trade-offs، ويتحمل مسؤولية النظام في Production. كتابة الكود جزء من شغله، لكنها مش تعريف الشغل كله.



ليه تعريف Software Engineer بقى محتاج يتراجع؟

المشكلة

لفترة طويلة، كتابة الكود كانت أكثر جزء ظاهر من شغل مهندس البرمجيات. ومع الوقت، ناس كتير بدأت تعتبر إن ده هو تعريف الشغل كله.

النهارده الـ AI خلى كتابة الكود أسرع بكثير — فبقى واضح إن دور المهندس أكبر من التنفيذ نفسه.

مهندس البرمجيات **بيسأل الأسئلة الصح**
قبل ما يكتب الكود.
بيفهم المشكلة، وبيختار الحل المناسب للواقع.

إحنا بنحل إيه أصلاً؟

إيه البنية المناسبة للحل؟

الحل ده هيكلفنا كام؟

هل الحل ده Reliable؟

قابل أحمد... وباسل.

الأتين بيعرفوا يكتبوا كود، والاتين ممكن يستخدموا AI. الفرق يبدأ من أول خطوة في التفكير.

أحمد

- يركّز على التنفيذ
- يحب يشوف النتيجة بسرعة.



طريقته في البداية

“إيه المطلوب؟
وأكتبه إزاي؟”

باسل

- يسأل قبل التنفيذ
- يحاول يشوف الصورة كاملة.



طريقته في البداية

“إحنا بنحل إيه؟
وليه ده الحل المناسب؟”

نفس القدرة على كتابة الكود. نقطة البداية مختلفة.

أحمد

باسل

“هنكتبها إزاي؟”	أول سؤال	“إحنا بنحل إيه أصلاً؟”
يبدأ في التنفيذ مباشرة	قبل الكود	يفهم القيود وال assumptions قبل التنفيذ
ال feature اشتغلت وال tests عدّت	تعريف النجاح	الحل صحيح، قابل للتشغيل، ومناسب للواقع
يسرّع كتابة ال implementation	استخدام AI	يسرّع التنفيذ ثم يراجع التصميم والنتيجة
يصلح المشكلة اللي ظهرت	وقت الفشل	يسأل: ليه حصلت؟ وإيه ال failure mode؟
يفكر لحد تسليم ال task	أفق التفكير	يفكر في ال Production والتغيير مع الوقت

الفرق الحقيقي مش في الكود. الفرق في التفكير قبل الكود ، والمسؤولية بعد ما يدخل Production .

وده هو السؤال اللي محتاجين نجابو عليه:

إيه اللي يفرّق مهندس البرمجيات عن مجرد كتابة الكود؟



إزاي تبقى زي باسل؟



CODING $\neq$ SOFTWARE ENGINEERING

الكود ينفذ جزء من الحل. هندسة البرمجيات تختار الحل وتحمّل نتيجته.

عشان أفكر كمهندس برمجيات.

من هنا الفكرة تتحول لمنهج عملي. مش مطلوب تبقى متخصص في كل حاجة، لكن محتاج أساس يخليك تفهم القرار وتعرف إمتى تحتاج عمق أكثر.

05

Security

مين مسموح له يعمل إيه؟
وإزاي نحمي البيانات وال secrets؟

01

Problem & Requirements

إحنا بنحل إيه؟ لمين؟
وإيه القيود اللي لازم الحل يحترمها؟

06

Performance & Cost

الحل سريع كفاية؟ ينفذ يكبر؟
وبتكلفة منطقية؟

02

System Design

النظام يتقسم إزاي؟
وإيه ال boundaries وال trade-offs المناسبة؟

07

Production Engineering

بعد ال deploy، هنعرف مينين إن النظام شغال ونفهم
سبب أي مشكلة؟

03

Data & Databases

البيانات شكلها إيه؟
وإزاي نخزنها ونغيرها من غير ما نفقد صحتها؟

08

Testing & Change

هل نقدر نغير النظام ونطلق versions جديدة من غير ما
نكسر الموجود؟

04

Reliability

إيه اللي يحصل لو dependency وقعت؟
وال request اتكررت، أو حمل النظام زاد؟

إحنا بنحل إيه أصلاً؟

اتعلم إيه؟

Requirements

Use cases

Functional vs Non-functional

Constraints

Edge cases

Business context

قبل ما تختار Framework أو Database، لازم تكون فاهم المستخدم، القيمة المقدمة، وإيه اللي يعتبر نجاح فعلاً.

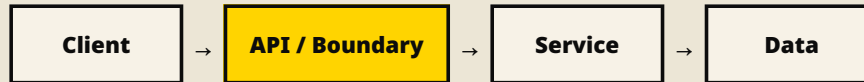
عملياً تعمل إيه؟

قبل أي Feature، اكتب أربع حاجات:

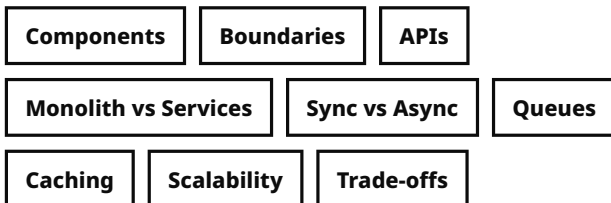
- المشكلة
- المستخدم
- معيار النجاح
- القيود

متبنيش حل ممتاز لمشكلة محدش محتاجها.

النظام ده المفروض يتبني إزاي؟



اتعلم إيه؟



اسأل قبل التنفيذ

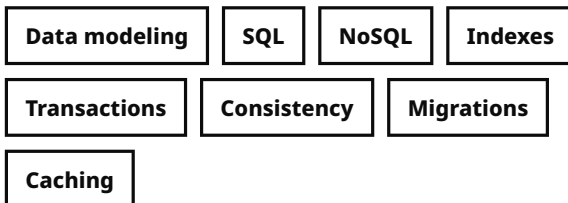
- فين ال boundaries؟
- إيه ال bottleneck المتوقع؟
- إيه أبسط Design يكفي دلوقتي؟
- إيه القرار اللي هيكون غالي تغييره بعدين؟

صمم لاحتياج النهارده، بس خليه سهل يتوسّع ويتغير بعدين من غير Refactoring كبير.

البيانات دي هتعيش فين وإزاي؟



اتعلم إيه؟



رگز على القرارات

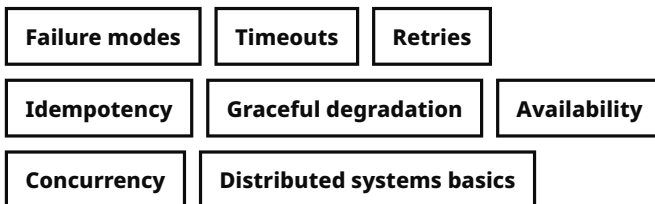
- إيه اللي بيتقرأ كتير؟
- إيه اللي بيتكتب مع بعض؟
- هل لازم التغيير يبقى atomic؟
- إيه اللي يحصل وقت migration؟

البيانات هي حالة ال **System**. وأي قرار غلط في ال **Data Model** ممكن يبقى تغييره مؤلم بعدين ويتطلب **Migrations** صعبة. عشان كده، خد وقتك قبل أي تغيير في شكل البيانات.

ال Happy Path مش هو ال Production.



اتعلم إيه؟



السؤال الهندسي

- لو جزء من النظام فشل، هل باقي التجربة تفضل مفهومة؟
- هل نقدر نعيد المحاولة من غير ما نكرر أثر العملية؟

الفشل مش هتمنعه 100%، لكن دور على ال edge cases وال failure modes بدري عشان تقلل احتماله وتأثيره قبل ما يوصل للمستخدم.

مين مسموح له يعمل إيه؟

مش مطلوب كل مهندس يبقى Security Specialist. لكن أي Software Engineer لازم يعرف يبني Defaults آمنة ويفهم المخاطر الأساسية.

Identity & Access

Authentication

Authorization

Least privilege

Input & Data

Validation

Data exposure

Encryption basics

Secrets & Infrastructure

Secrets

Permissions

Secure defaults

Web Risks

OWASP basics

Injection

Session risks

Security مش Feature تتضاف في الآخر.
هي جزء من التصميم.

الحل شغال... بس هل ينفع يكبر؟

Speed

Latency

Scale

Capacity

Cost

Resources

اتعلم إيه؟

Latency

Throughput

Bottlenecks

Profiling

Caching

Capacity

Cloud resources

Cost awareness

التحسين يبدأ بالقياس

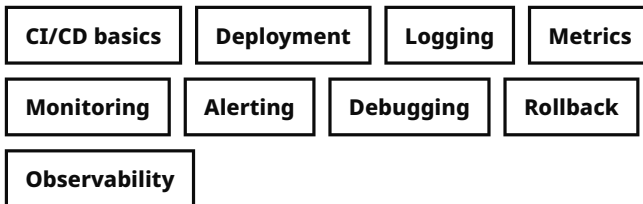
- حدّد الـ bottleneck الحقيقي قبل أي optimization.
- قس الأداء قبل التغيير وبعده.
- قيم هل التحسن يبرّر التعقيد والتكلفة الإضافية.

الأداء قرار تقني واقتصادي: أسرع أو أرخص — من غير تعقيد مالوش لازمة.

ال Deploy بداية مسؤولية جديدة.



اتعلم إيه؟



لازم تعرف تجاوب

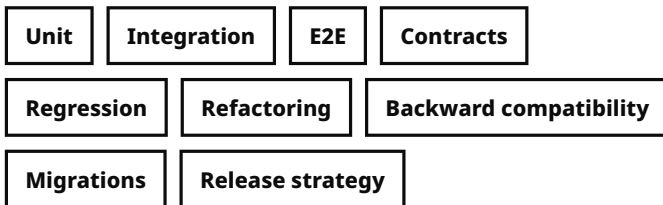
حصل إيه؟ لمين؟ من إمتى؟ فين؟ وإيه الدليل
اللي يثبت السبب بدل التخمين؟

القيمة: لما يحصل Incident، تقلل وقت التخمين وتوصل للسبب أسرع.

هل أقدر أغَيّر النظام من غير ما أكسره؟



اتعلم إيه؟



الفكرة الأهم

الـ Tests مش هدفها تثبت إن الكود شغال النهارده فقط. قيمتها الأكبر إنها تخليك تغيّره بكرة بثقة أعلى.

الهندسة الجيدة تقلّل تكلفة التغيير.

16

وال AI يدخل فين؟

ال AI يقدر يسرّع كل Area تقريبًا. لكن السرعة مش معناها إن مسؤولية القرار اختفت.

AREA	ال AI يسرّع	المهندس يمتلك
Problem	تلخيص Context وصياغة أسئلة	تحديد المشكلة اللي تستحق الحل
Architecture	اقتراح Designs وبدائل	اختيار ال Trade-offs المناسبة
Data	Migrations و Queries و Schemas	صحة ال Model وال Consistency
Reliability / Security	اقتراح Tests ومخاطر محتملة	تقدير الخطر والقرار النهائي
Production	تحليل Logs وشرح Signals	Incident judgment والمسؤولية
Testing	توليد Test cases بسرعة	تحديد إيه اللي لازم يتختبر فعلاً

كل ما ال AI يسرّع التنفيذ، قيمة ال Engineering Judgment بتزيد.



17

أدرسهم بأي ترتيب؟

لو أنت عارف Syntax و Framework و بتبني Features، امش بالترتيب ده — لكن طَبِّق على مشروع حقيقي من أول مرحلة.

01

Data + HTTP + API fundamentals

Data modeling · SQL · HTTP · API boundaries · transactions

02

Testing + Debugging

Tests · logs · reproducing bugs · reading failures systematically

03

System Design

Boundaries · queues · caching · consistency · trade-offs

04

Production + Observability

Deploy · monitoring · metrics · incidents · rollback

05

Security

AuthN/AuthZ · validation · secrets · permissions · web risks

06

Performance + Scale + Cost

Latency · bottlenecks · capacity · cloud cost · optimization

Problem framing مش مرحلة منفصلة. خليه عادة في كل Project: بنحل إيه؟ ليه؟ وإيه القيود؟

متستناش تخلص الـ Roadmap. ابني، Deploy، وخلي كل مشكلة تقابلك تحدد موضوع الدراسة الجاي.

18

Software Engineer Checklist

قبل ما تعتبر Feature "خلصت"، جرب تجاوب على الأسئلة دي. الصفحة دي معمولة
عشان ترجع لها كل مرة.

قبل ما تكتب الكود

- | | | | |
|--------------------------|--|--------------------------|---|
| ☐ | إيه ال constraints وال success criteria؟ | ☐ | إحنا بنحل إيه؟ ومين المستخدم؟ |
| ☐ | إيه أبسط Design مناسب دلوقتي؟ | ☐ | البيانات شكلها إيه؟ وإيه ال consistency المطلوبة؟ |

قبل ال Deploy

- | | | | |
|--------------------------|--|--------------------------|---|
| ☐ | إيه ال security risk وال permissions؟ | ☐ | إيه اللي ممكن يفشل؟ وهل العملية Idempotent؟ |
| ☐ | ال scale وال latency وال cost مقبولين؟ | ☐ | هل ال tests بتحمي أهم behavior؟ |

بعد ال Deploy

- | | | | |
|--------------------------|---|--------------------------|---------------------------------|
| ☐ | لو حصل Incident، هل عندنا Logs / Metrics كفاية؟ | ☐ | هنعرف مين إن النظام شغال فعلاً؟ |
|--------------------------|---|--------------------------|---------------------------------|

أي سؤال مش عارف تجاوبه = موضوع دراسة أو Experiment حقيقي.

شكراً

مع تمنياتي بالتوفيق

الـ AI مش بديل عنك — هو Amplifier
للتفكير، و Accelerator للتنفيذ.